

Shopping Copilot

TECHNICAL DOCUMENT

Live demo: [Shopping Copilot](#)

1. Project overview

Shopping Copilot is a conversational product-selection assistant that combines retrieval, neural reranking, reversible requirement memory and evidence-based product comparison. Four modules contribute distinct capabilities: Search/Ranking finds candidates, the Agent maintains intent and coordinates actions, Description/Comparison analyzes selected evidence, and the Frontend supports inspection and confirmation.

Repository: [productSelectionAgent](#). Implementation reference: `main` commit `1abcb6c`. Earlier measurements retain their original commit and environment labels.

2. Executive summary

A product can be relevant to a query without being ready for a decision. Shopping Copilot connects search, requirement refinement and evidence-based comparison so that shoppers can inspect their options before confirming a selection. Its intended value is less manual cross-checking and clearer reasons for a choice. Reported held-out ranking nDCG improves from 0.859077 to 0.867305, while 615 isolated Agent regressions pass on the documented commit. Evaluation scope and remaining gaps are summarized in Section 8.

3. Goal, scope and design choices

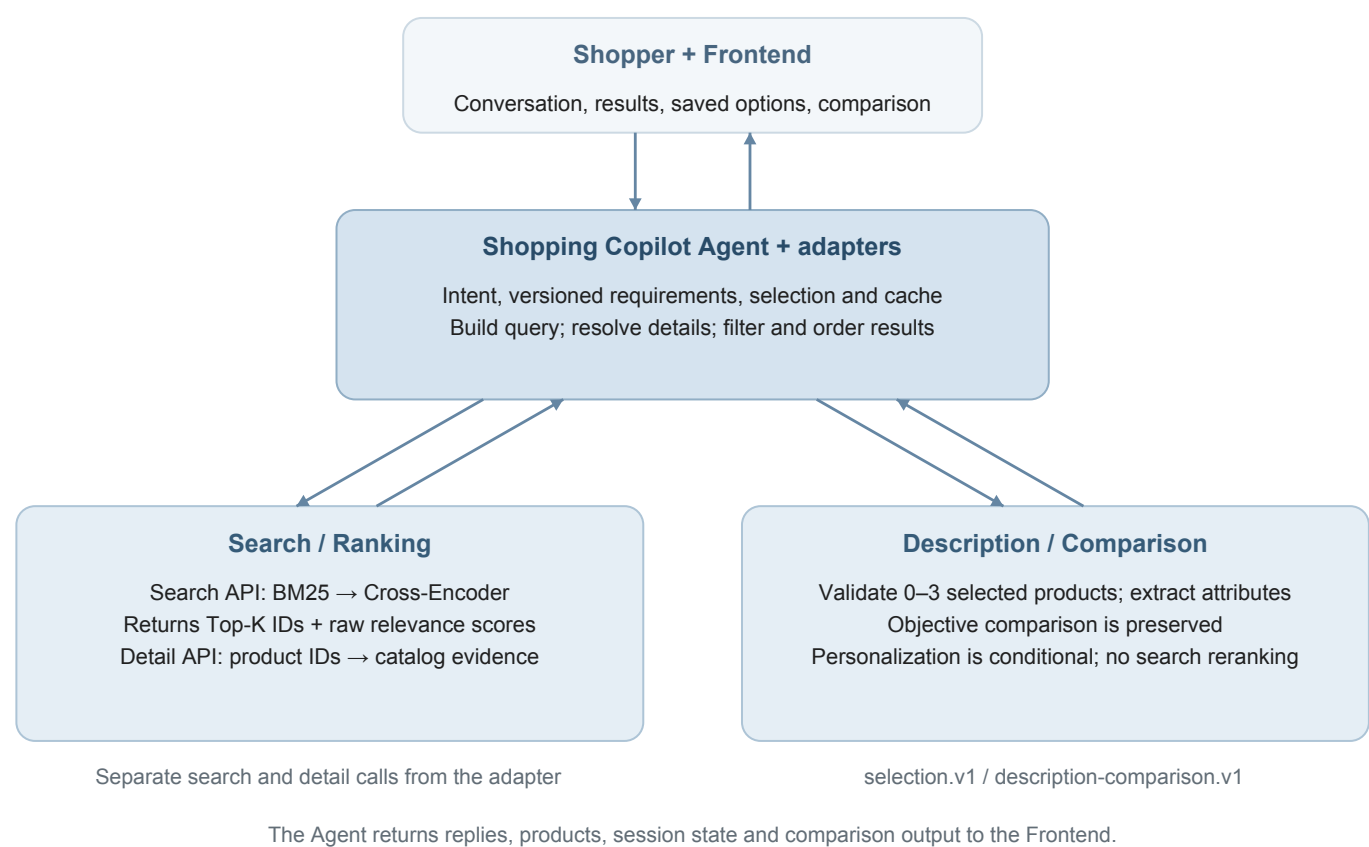
The goal is to help a shopper move from a revisable natural-language request to an informed selection of catalog products. The prototype supports search, refinement, product questions, a shortlist of up to three items and explicit confirmation. It does not place orders, process payments or verify live inventory.

Decision bottleneck	Design response	Deliberate trade-off
Large catalogs make exhaustive neural scoring expensive	BM25 followed by Cross-Encoder reranking	Reranking cannot recover candidates missed by retrieval
Preferences change across turns	Versioned requirements, stable IDs and Undo/Redo	Requires explicit state and reference handling
Relevant products are difficult to compare	User-selected set and category-aware attribute profiles	Comparison is bounded by supplied catalog evidence
Personalized advice can distort factual claims	Objective and personalized stages with separate inputs	Sequential generation adds latency; quote matching is weaker than semantic verification
Missing data and uncertain requests can mislead	Unknown/partial states, visible evidence and explicit confirmation	Some requests need clarification or recovery

4. Architecture and reasoning loop

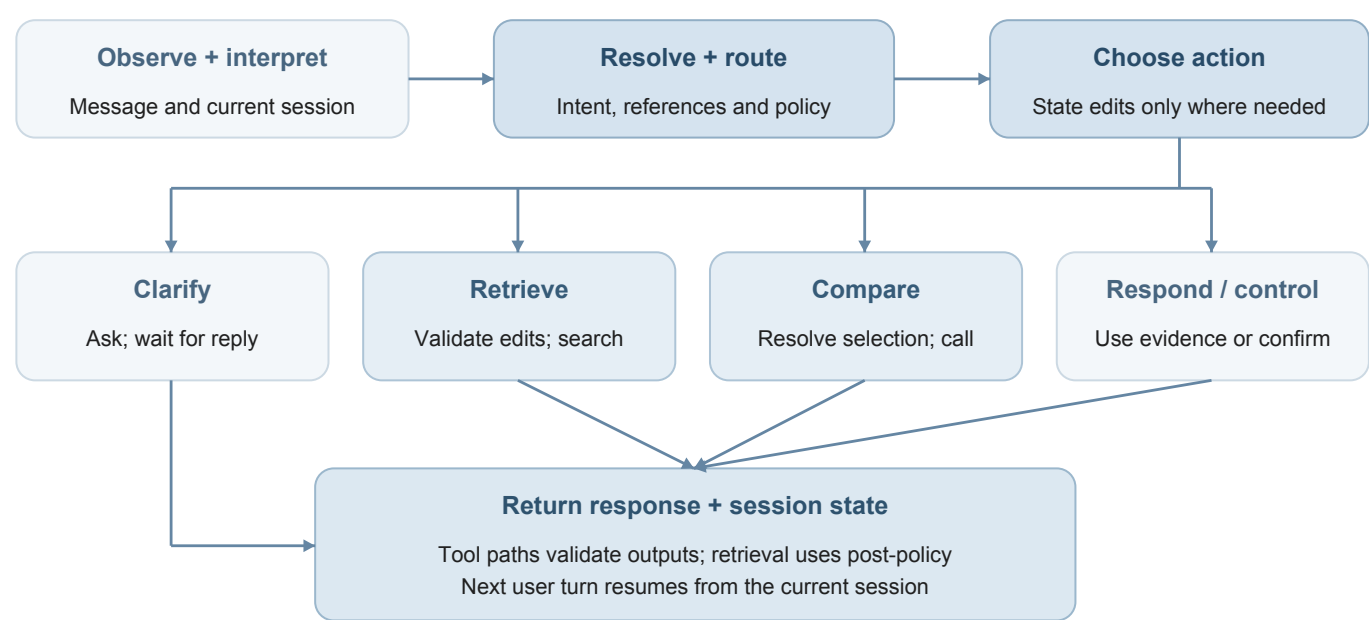
Shopping Copilot uses one coordinating Agent with specialized tools and modules. Session requirements and selections belong to the runtime; catalog relevance belongs to Search/Ranking; comparison evidence belongs to Description/Comparison; interaction and rendering belong to the Frontend.

Figure 1. Component responsibilities and data flow



The Agent uses a bounded interpret-update-plan-act-observe loop. It can clarify, retrieve, answer from existing evidence or compare resolved products. Optional model assistance proposes validated edits and response text; explicit policies retain control of actions.

Reasoning loop and human clarification checkpoint



The loop makes new evidence and state changes available to the next turn. Read-only questions can reuse the current view; ambiguous references require clarification. Section 5 gives each module's implementation and evidence.

5. Technical methods by module

5.1 Search and Ranking

Owner: Search/Ranking.

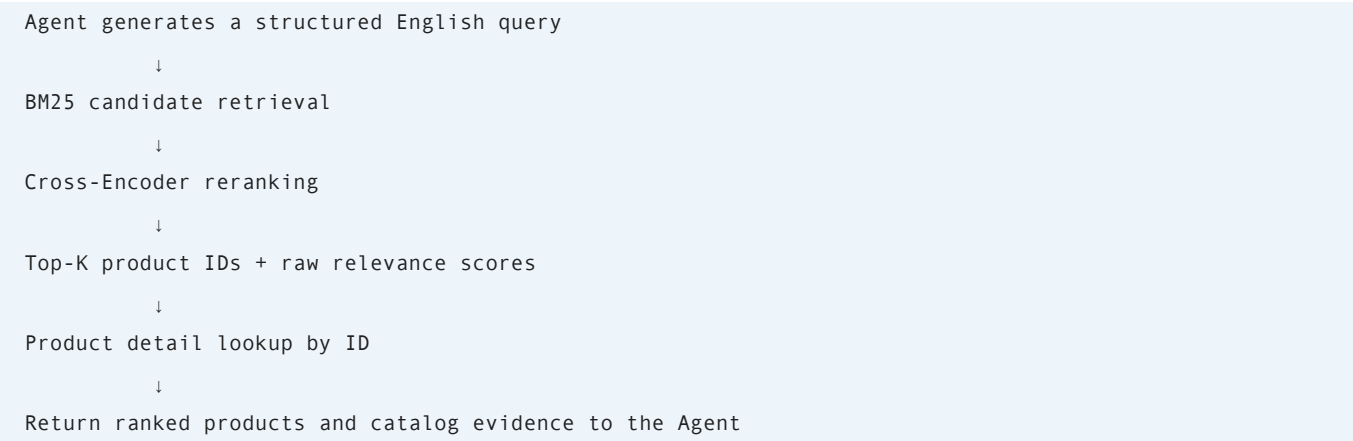
The Search & Ranking module focuses on the key elements of user requirements and translates those requirements into relevant product results. During retrieval and ranking, it identifies products that match what the user is looking for and prioritizes those with a stronger match. For example, for the request “a blue cotton dress,” the system needs to consider product category, color, and material together, so that the returned results reflect these elements rather than merely matching isolated query terms.

This task requires balancing search efficiency with matching quality. On the one hand, the product catalog is large, so the system must quickly narrow the search space to reduce subsequent computation. On the other hand, user queries and product descriptions may express the same information differently, making term overlap alone insufficient to fully assess their relevance. Although neural networks can further model the semantic relationship between queries and products, scoring every product in the catalog would incur substantial computational cost. Efficient retrieval therefore needs to be combined with fine-grained ranking.

To address this, we adopt a two-stage retrieval and ranking architecture: BM25 first retrieves candidate products from the catalog efficiently, and a Cross-Encoder then jointly models the query and candidate product text to rerank the candidates by relevance. The system subsequently retrieves the corresponding catalog information using the IDs of the top-ranked products and returns both the ranking results and product evidence to support the Agent’s decisions and subsequent product comparison.

5.1.1 Inputs and workflow

Search & Ranking follows a “candidate retrieval - semantic reranking - detail lookup” architecture. The Agent formulates the current user requirements as an English query, BM25 retrieves candidate products, and the Cross-Encoder rescores and reranks them. Catalog information is then retrieved by product ID to provide evidence for downstream explanations and comparisons.



The module takes English query text and a maximum number of results as input, and outputs product rankings, relevance scores, and available catalog information. The data flow between stages is as follows:

Data flow direction	Data transferred
Agent → Search module	English query query and result limit top_k
BM25 → Cross-Encoder	Candidate product set and corresponding product text
Cross-Encoder → Detail lookup stage	Top-K product IDs, with raw scores retained for association with the results
Search module → Agent	Ranked products, raw relevance scores, and available catalog evidence

Search and detail lookup are connected through two interfaces:

```
search_products(query, top_k)
get_product_details(product_ids)
```

`search_products` returns product IDs and scores ordered by relevance. The Agent then passes these IDs to `get_product_details` to retrieve product information in the same order, forming a complete result containing both rankings and evidence.

We use Amazon’s **Shopping Queries Dataset (ESCI)** to train and evaluate the product ranking model. Each sample represents a “user query-product” pair, with a human-annotated label indicating how the product matches the query. [Official dataset documentation](#)

The dataset primarily contains three types of information:

- **Query information:** Query text, query ID, locale, and training/test split.
- **Product information:** Product ID, title, description, bullet points, brand, and color.
- **Relevance labels:** Exact (an exact match), Substitute (a substitute), Complement (a complementary product), and Irrelevant (unrelated).

This project uses the dataset’s ranking-task subset, with the following scale:

Data split	Number of queries	Number of annotated query-product pairs
Training set	20,888	419,653
Test set	8,956	181,701
Total	29,844	601,354

5.1.2 Core methods

BM25 is a term-based text retrieval algorithm that assigns relevance scores to products by measuring the occurrence of query terms in product text. It combines term discrimination, term-frequency saturation, and text-length normalization to give greater weight to distinctive query terms while preventing repeated terms or longer descriptions from disproportionately influencing the ranking.

In product search, queries typically contain key elements such as product category, brand, color, or material. BM25 can use these terms to quickly locate relevant products, making it suitable as the first stage of large-scale catalog search. We use BM25 to narrow a catalog of over one million products to a bounded candidate set, allowing the subsequent Cross-Encoder to focus its computation on products more likely to meet the user’s requirements.

The standard BM25 scoring formula is:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}$$

Here, $f(t, d)$ denotes the number of occurrences of query term t in product text d , $\text{IDF}(t)$ measures the term’s discriminative power, and $|d|/\text{avgdl}$ is used for text-length normalization. The parameter k_1 controls term-frequency saturation, while b controls the strength of length normalization.

Our implementation uses the Lucene method in `bm25s`, with $k_1 = 1.5$ and $b = 0.75$. The processing workflow is as follows:

```

US product data
↓
Remove invalid IDs, duplicate products, and empty text
↓
Concatenate product titles and bullet points, then tokenize
↓
Build the BM25 index (1,215,854 products)
↑
English query → Process with the same tokenization method
↓
Retrieve and retain products with valid matches
↓
Up to 100 candidate products → Cross-Encoder reranking

```

The index and queries use the same tokenization method, and stopwords are retained to avoid removing query information such as `no` and `without` during preprocessing. By default, the retrieval stage selects up to 100 candidate products, filters out results without valid term matches, and passes the product text to the reranking model. The product table and index are saved together to ensure that retrieval results map accurately to the corresponding products.

A Cross-Encoder is a relevance model that jointly models a query and product text. It feeds both texts into a Transformer simultaneously, allowing the requirements in the query and the information in the product description to interact directly through the attention mechanism. It ultimately outputs a score representing their degree of match.

This approach is suitable for fine-grained product ranking: users typically express their requirements briefly, while relevant product information may be scattered. The Cross-Encoder can combine this information to assess the overall match and use context to model expressions with different wording but similar meanings. Since BM25 has already narrowed the candidate set, this stage can focus on the detailed relationships between the query and candidate products.

We concatenate the information into product text x_d , then combine it with query q to form the input:

```

[CLS] User query [SEP] Product Info [SEP]
      ↓
Transformer joint encoding
      ↓
Relevance scoring head
      ↓
Scalar score  $s(q,d)$ 

```

This process can be expressed as:

$$h_{q,d} = \text{Transformer}_{\theta} ([\text{CLS}], q, [\text{SEP}], x_d, [\text{SEP}])$$

$$s(q, d) = \text{Head}_{\theta} \left(h_{q,d}^{[\text{CLS}]} \right)$$

Here, $h_{q,d}^{[\text{CLS}]}$ is the summary representation obtained by jointly encoding the query and product, which the scoring head maps to a single relevance score. Because the two texts interact during encoding, the model can learn how different elements of the query correspond to the product description, rather than merely comparing the similarity of two independently generated text representations.

We start from the pretrained model `cross-encoder/ms-marco-MiniLM-L-12-v2` and fine-tune it using annotated query-product pairs from ESCI, adapting the model to relevance assessment in product search.

During training, each sample consists of a user query, a product title and bullet points, and the corresponding ESCI label. We map the four labels to graded relevance targets:

Label	Relevance meaning	Target gain
Exact	Exactly matches the requirements	1.00
Substitute	Can serve as an alternative	0.10
Complement	Complements the target product	0.01
Irrelevant	Unrelated to the query	0.00

This configuration makes the training objective reflect the priorities of product search: exact matches should rank above substitutes, substitutes above complementary products, and irrelevant products receive the lowest target gain.

The model is trained using pointwise regression, with mean squared error loss reducing the difference between predicted scores and target gains:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (s(q_i, d_i) - g_i)^2$$

Here, N is the number of samples included in the calculation, and g_i is the target gain for the i -th query-product pair. Training updates the model parameters through backpropagation, progressively adapting the scores to the relationships between query requirements and information such as product categories, attributes, and uses.

Training and development data are split by query, preventing different product pairs for the same query from appearing in both sets. During training, the development set is used to monitor ranking performance, and the final evaluation is conducted on the independent official test set. Training, testing, and online reranking all use the same “title + bullet points” product representation.

5.1.3 Outputs and downstream use

During online search, the Cross-Encoder scores each candidate product returned by BM25 and selects the Top-K products in descending score order:

$$R_K(q) = \text{TopK}_{d \in C(q)} s(q, d)$$

Here, $C(q)$ is the candidate set generated by BM25. The module returns **product IDs and raw scores** ordered by relevance. Product details are then retrieved using these IDs, providing the Agent with ranking results and the corresponding catalog evidence.

Raw scores are used to compare the relevance of candidate products for the same query; they are not calibrated probabilities. Through this stage, the system further organizes the term-relevant candidates found by BM25 into a product list that accounts for context and semantic matching, prioritizing options that more closely align with the user’s requirements.

5.1.4 Evaluation and limitations

We evaluate the model’s ranking performance using the official ESCI US test set, covering **8,956 queries and 181,701 query-product pairs**. For each query, the model scores the given candidate products individually and ranks them in descending score order. Before evaluation, we check the predicted query-product pairs against the test set to ensure that no candidates are duplicated, omitted, or added.

The primary evaluation metric is **full-list nDCG**. It considers both product relevance and ranking position: placing exact matches higher produces a higher ranking score. We retain the ESCI gains used during training, mapping Exact, Substitute, Complement, and Irrelevant to 1.0, 0.1, 0.01, and 0.0, respectively, and calculate:

$$DCG(q) = \sum_{i=1}^{n_q} \frac{g_i}{\log_2(i+1)}$$

$$nDCG(q) = \frac{DCG(q)}{IDCG(q)}$$

Here, g_i is the relevance gain of the product at position i , and $IDCG(q)$ is the score of the same candidate set under an ideal ranking. The final result is the mean nDCG across all test queries, allowing ranking quality to be compared on a common scale across different queries.

We conducted multiple ablation comparisons of product text representations to examine how different information combinations affect ranking quality, ultimately selecting **Title + Bullet Points** as the product input. Titles provide product categories and core identifying information, while bullet points add details such as materials, functions, and usage scenarios. Together, they provide a more complete textual basis for matching queries to products.

The verifiable baseline and final-system results from the saved prediction records are as follows:

Approach	Number of test queries	Full-list nDCG
Baseline model	8,956	0.859077
Title + Bullet Points	8,956	0.867305

The final approach improves over the baseline by **0.008228 nDCG**. This result supports our adoption of a product representation combining titles and bullet points, used consistently for model training, online reranking, and retrieval index construction.

The reported test evaluates reranking over the supplied ESCI candidates. It does not measure BM25 recall over the full catalog or end-to-end shopping performance; relevant products missed during candidate retrieval cannot be recovered by reranking.

5.2 Shopping Copilot Agent and orchestration

Owner: Agent.

The Agent turns a sequence of shopping messages into a consistent, revisable decision process. A relevant result list alone cannot interpret “blue instead,” retain a saved product after the list changes, or distinguish a product question from a new preference. The Agent therefore maintains explicit state and selects the next permitted action from that state. Its contribution is continuity and controlled execution, rather than an unrestricted model that decides every action from the conversation transcript.

5.2.1 Inputs and workflow

Each turn begins with the message and current session context. The runtime distinguishes read-only questions and conversational controls from requirement changes. The intent router and structured memory manager interpret supported operations, apply corrections and export the resulting requirements with a session ID, turn and state version. Source text and source turns support an inspectable account of what changed.

The implementation separates two responsibilities. `FinalAgent` in `shopping_agent/agent.py` coordinates intent interpretation, structured memory, pre-retrieval policy, retrieval/ranking and post-retrieval policy. `AgentRuntime` in `mvp/server.py` adds the user-facing session lifecycle, product-reference handling, shortlist, comparison handoff, finalization and HTTP responses.

State group	Representation and purpose
Hard requirements	Explicit necessary conditions, such as category or a strict budget; unsupported evidence must not become a verified match
Soft preferences	Weighted acceptable choices that may influence ordering without excluding every alternative
Exclusions and product feedback	Rejected attribute values and stable product IDs; hiding an item is distinct from removing it from the shortlist
Conversation context	Pending clarification, displayed IDs, browsing/guidance mode and deferred category details
Decision context	Saved product records, comparison scope and finalization status
History and provenance	Source turns, requirement evidence, state versions and snapshots used by correction and Undo/Redo

For example, “I prefer cotton, but polyester is okay” keeps a first choice and fallback; it is not equivalent to a cotton-only requirement. Product IDs retain identity while display ranks change. Category-specific details can be deferred, but the current search explores one category at a time. The web service stores sessions in memory with defaults of 128 sessions and a 3,600-second idle lifetime. These limits do not establish concurrent-user capacity, durable storage or account-wide personalization.

Search is needed when an active request requires new candidates, including relevant requirement changes or requests for more options. The `SearchToolAdapter` translates structured requirements into an English query and calls `search_products(query, top_k)`. It then requests records through `get_product_details(product_ids)`. The shared tool owns BM25 retrieval and local model scoring. The adapter validates IDs and finite scores, checks catalog-supported requirements and exclusions, and applies supported soft preference ordering. The displayed batch is capped at ten products. Raw search scores remain relevance values, not probabilities or quality certificates.

Retrieval and ranking contracts carry the session, turn and state version; context validation rejects stale or cross-session results. Candidate limits, unique IDs and rank continuity are checked. The current catalog adapter leaves price and ratings unknown. It cannot certify a strict budget from those records, even when textual relevance is high.

Read-only product questions and preference recaps reuse session/catalog context instead of launching a fresh search. Comparison is a separate branch: the runtime resolves the exact requested IDs and order, or uses saved IDs when the shopper requests saved options. The description adapter builds `show-me-your-agent.selection.v1`, including current requirements and bounded product sources, and invokes the module described in Section 5.3. Comparison does not rerank the search list. An explicit comparison group and the saved shortlist are separate concepts; finalization and export refer to saved options.

The runtime caches comparison results by product identity/order, requirements, supplied catalog facts and provider identity. Changed inputs invalidate reuse. Repeated completed comparisons, export and finalization can reuse available output; explicit retry can replace a partial result. Retry retains resolved product IDs rather than interpreting old screen ranks against a newer page. Cache hits avoid additional comparison model calls, but are not evidence of lower end-to-end latency across an unmeasured workload.

5.2.2 Core methods

Compound requests retain separately stated constraints. Category changes replace the active search context; deferred details remain associated with their intended category. If multiple categories need disambiguation, the Agent asks which to explore first rather than implying that both were searched. An explicit correction takes precedence over the earlier value. Tentative preferences remain distinguishable from firm requirements.

Displayed references are resolved before a combined edit can refresh the results. Consequently, “How much is #2? I prefer cotton” refers to the second item the shopper was actually viewing. Invalid or ambiguous references trigger a bounded clarification. Some ambiguous comparison requests defer the accompanying edit until valid references arrive; the pending edit is not presented as an already saved preference.

Rules and explicit state operations remain the fallback interpretation path. When DeepSeek is configured, `PrimaryRequirementParser` receives the current message, current hard/soft/excluded state, any pending question and deterministic rule hints, then returns only the edits requested in that turn. A fully valid batch atomically replaces the rule edits; a network error, unsupported operation, missing source quote or ungrounded value falls back to the deterministic path without partially mutating state. Other providers retain the earlier supplementary `requirement_enhancer` behavior, and model assistance remains off unless a provider is explicitly configured.

The state now includes an explicit audience dimension for men, women, boys, girls, kids, baby and unisex. Audience is recorded only from explicit user language, is retained when the shopper switches product category and is enforced against catalog department, title or category evidence. The Agent does not infer audience from terms such as “dress” or from a size. Unknown or contradictory labels therefore cannot silently satisfy a hard audience constraint. Coverage remains limited to implemented and tested language patterns rather than arbitrary natural-language understanding.

The planning pattern is a bounded, stateful policy loop. Pre-retrieval policy determines whether the current context calls for clarification, retrieval or acknowledgement. Post-retrieval policy inspects returned candidates and decides whether an evidence-backed follow-up would help. Control paths can answer directly without entering search.

Situation	Implemented policy	User-visible effect
Optional refinement	Use catalog-supported distinctions and estimated candidate reduction, with an attention budget	Ask a useful question alongside results rather than enforce a fixed questionnaire
Browsing or “show me first”	Respect the request to see results and dismiss optional narrowing where appropriate	Exploration can continue without inventing a preference
Missing category or unverifiable strict budget	Request the necessary choice or explain the evidence gap	Skipping an optional question does not silently relax a hard condition
Product question	Resolve the displayed or contextual ID and answer from available listing evidence	Keep the current page and distinguish missing data from negative product evidence
“Show me more” or rejection	Prefer unseen eligible IDs and honor exclusions	Do not claim repeated results are new when the retrieved pool is exhausted
Undo/Redo	Restore the relevant saved state/history, including the applicable result view	Corrections remain reversible without rebuilding the request
Failed search or partial comparison	Surface the failure and provide explicit recovery where supported	Avoid an unbounded automatic retry loop
Save or finalize	Enforce the three-item shortlist limit and require an explicit finalization command	A comparison or generic acknowledgement is not a purchase authorization

The Agent can choose useful information-gathering steps within this action set. It does not place orders, make payments, reserve stock or operate an implemented staff approval service. A change to requirements or saved selection can invalidate a prior confirmation, requiring the shopper to review the current decision again.

5.2.3 Outputs and downstream use

The chat response gives the Frontend an assistant message, ordered products, a receipt describing requirements and decisions, selection state and, when applicable, a comparison handoff. Receipts expose such fields as `hard`, `soft`, `excluded`, `state_version`, `requirement_evidence`, `policy_reasons`, `suggested_replies`, `result_counts` and `model_usage`. The comparison result is attached under `comparison_assist`; the Frontend determines which returned fields are visible in its decision view.

When DeepSeek is enabled, a bounded `ResponseWriter` call can replace the conversational copy and add a short fit reason to each of at most ten displayed products. Candidate evidence is first extracted as short exact catalog excerpts; the model may select at most two validated excerpt IDs per product, but it cannot rewrite deterministic match signals, unknown/conflict warnings or a required clarification question. Unknown product IDs, invented evidence IDs, unsupported availability claims or malformed output trigger a fallback to the original deterministic response. `receipt.response_assist` records this stage and its usage, while `receipt.search_execution` separately reports whether the turn retrieved products, reset requirements or changed category.

The effect is a continuous, inspectable shopping session: the user can revise needs, inspect the same product, retain saved candidates and request a comparison without restating the whole task. Stable identity, explicit edits and confirmation boundaries prevent several tested forms of unintended state change. This establishes functional continuity; reductions in shopper effort, abandonment or staff workload still require user studies and a measured baseline.

5.2.4 Evaluation and limitations

On 28 September 2026, the isolated workflow regression suite passed **615 tests** on commit `1abcb6c`. These include synthetic catalogs and simulated model responses; the count is a software regression result, not a held-out language accuracy score.

Capability group	Evidence and checked behavior	Boundary
State correctness	Requirement, preference-scope, category, budget and audience cases; preserve hard/soft distinctions, explicit audience and applicable evidence	Coverage is limited to the test cases
Correction and recovery	Undo/Redo, reset, empty-pool and failed-search cases	Not an unrestricted recovery guarantee
Reference stability	Detail follow-ups, invalid ranks and mixed question/edit cases	Numbered references must resolve against a known display
Selection consistency	Shortlist, exact comparison scope and confirmation-boundary cases	No transaction system is exercised
Model integration	Atomic model-led requirement edits, deterministic fallback, grounded response-copy validation, simulated description success/failure, cache and retry cases	Does not measure live generative quality
Real-search lifecycle	<code>live_smoke</code> <code>--detail-followups</code> returned ten products, compared two, finalized the shortlist and passed audit checks	Uses the local search index; generative provider was off

The previously recorded real-search run on commit 62b6925 completed a five-turn detail-follow-up scenario, preserving product IDs and requirements through reference clarification, preference recap and a care question. Missing care instructions remained unknown. SHA-256 checks confirmed that all 26 files under the tested `search_tool` directory were unchanged by this run.

A separate local HTTP check on commit 62b6925 through the Vite proxy measured a **52.72-second cold search**, a **1.21-second warm search**, a **0.39-second comparison** and a **0.04-second finalization**. These are individual observations on Windows using Miniconda Python, PyTorch 2.5.1 with CUDA available, and cloud assistance off. Cold initialization exceeded the React client's 45-second timeout. The measurements are not p50/p95 estimates or controlled benchmarks; another search verification process was active during the cold check. Comparison used offline facts, not the three-stage generative pipeline.

Verified method-effect statement: explicit session state, bounded policies and versioned tool handoffs support tested correction, reference continuity and controlled selection from retrieval through finalization. Broader language robustness, live-model factuality, production throughput and user-effort improvements remain unmeasured.

5.3 Description and Comparison

Owner: Description/Comparison.

The Description/Comparison module helps shoppers understand the differences between products they have selected. Search may return several relevant options, but their descriptions often use different terms, emphasize different features or omit information needed for comparison. A shopper looking at three blue cotton dresses, for example, may still need to compare fabric composition, sizing and care instructions before deciding which one suits their needs.

The module organizes this information in two layers. Objective comparison describes what the available listings support, including differences and trade-offs. Personalized comparison relates those findings to the shopper's current requirements. Keeping these layers separate allows the user to distinguish a product fact from a reason that the fact matters to them. Both layers retain their supporting evidence and identify missing information.

5.3.1 Inputs and workflow

The module starts after product retrieval and selection. It receives the Agent's versioned `show-me-your-agent.selection.v1` handoff, containing up to three selected products in the requested order and the current requirements grouped as `hard`, `soft` and `excluded`. Each product has a unique, nonempty `parent_asin` identifier. Available factual sources include the title, description, bullet points, scalar detail fields, finite price and finite rating.

Input validation checks the schema, product count, identifiers and requirement types. An empty selection returns an empty result; one product can be profiled, while two or three can be compared. Duplicate identifiers and selections exceeding the limit are rejected. The Frontend's requirement to save at least two products before opening its comparison view is a separate interface rule.

The processing sequence consists of attribute extraction, objective comparison and personalized comparison. Schema selection prepares the first stage rather than adding a separate model call. With a provider configured, a nonempty selection normally requires three calls when requirements are active, or two when personalization is unnecessary. Without a provider, the module returns an offline view of direct facts.

The first two stages receive allowlisted product sources, excluding ranking scores, upstream recommendation text and selection rationale. User requirements are withheld from these stages, except for a category hint used to choose the attribute schema. The final stage receives the requirements together with the established objective information. This separation is enforced through the data passed to each call.

The Agent resolves product references and supplies the catalog records. The comparison module does not search the dataset, select alternative products or obtain missing listing information. It preserves the supplied IDs and order, allowing the Agent and Frontend to associate every profile, comparison cell and explanation with the

intended product.

5.3.2 Core methods

The extraction schema combines universal attributes, category-specific attributes and additional attributes discovered in the supplied evidence. Universal fields include brand, model, material, colour, size, dimensions, price, rating, care and warranty. Category-specific fields make the comparison useful for different product types: fabric composition and fit matter for clothing, while hardware specifications matter for laptops. Additional supported fields allow useful listing information to be retained even when it is outside the predefined schema.

The Agent's category is a hint for choosing dimensions, not evidence that every selected product belongs to that category. If no hint is supplied, the extraction stage may identify a category from product text. An uncertain or unsupported category uses the generic schema. The result records whether the schema came from the Agent hint, model fallback or generic fallback, so a category assumption remains traceable.

The first model call extracts attributes from the allowed source fields. For each proposed value, it returns a source field, supporting quote and explicit/inferred label. The pipeline checks that the cited field exists and that the quote appears within it after normalization. Invalid references are rejected and reported as warnings. Product text is passed as untrusted data, with instructions that it must not change the extraction task.

Each schema field begins as unknown. A missing warranty statement, for example, does not establish that a product has no warranty. Direct catalog values for brand, colour, finite price and finite rating remain usable without model generation. The resulting product profiles combine comparable attribute names with the source information needed to inspect each accepted value. Citation validation establishes where an explanation points; it does not by itself prove that the interpretation of the quoted text is correct.

The second model call produces an objective assessment from the validated profiles and factual sources. It generates evidence-linked pros and cons and cross-product trade-offs. Alongside this prose, the pipeline builds a deterministic product-by-attribute matrix from the accepted profiles and lists unknown attributes. The output sets `absolute_winner`: `null`, since different supported characteristics need not produce one universally preferable option.

Every generated comparison point must cite source evidence for the relevant product. Unknown information is kept separate from an established disadvantage. For example, an absent care instruction should be reported as information to check, rather than used to conclude that a garment is difficult to maintain. This distinction is particularly useful when selected listings differ in completeness.

The first two stages form the module's **preference firewall**: factual extraction and assessment are not directly conditioned on the shopper's preferences, with the category hint retained as the schema-selection exception. Changing a preference should therefore change the interpretation of product differences, not the underlying evidence. This is an input-isolation property; separate live model calls may still produce different wording or extraction choices.

The third call receives the objective information and current hard requirements, soft preferences and exclusions. It returns `fit_reasons` and `cautions` for each product, supported by evidence references. A difference in material, for instance, can be relevant to a stated fabric preference without being an objective advantage for every shopper. The model is instructed not to claim that an unsupported constraint, such as a budget with no available price, has been satisfied.

Personalization preserves the selected order and sets `ranking_changed`: `false`. It explains the existing selection rather than replacing the Search/Ranking result. When there are no active requirements, the module skips this call and retains the objective result. This avoids generating a personal recommendation without a stated basis.

5.3.3 Outputs and downstream use

The result follows the `description-comparison.v1` schema and keeps objective and personalized information separately accessible. The main fields support both a detailed comparison view and the existing shortlist interface.

Output	Purpose
Selected IDs and product profiles	Preserve product identity and order; expose attributes, values and source evidence
Objective comparison	Provide the attribute matrix, per-product assessments, trade-offs and unknowns
Personalized comparison	Provide requirement-specific fit reasons and cautions without changing order
Compatibility projection	Supply up to three evidence-linked pros and cons per product
Execution metadata	Record status, warnings, provider/model, token usage, total latency and provenance

The Agent invokes `DescriptionComparison(provider).compare(handoff).to_dict()` and attaches the returned dictionary under `comparison_assist`. Backend serialization makes it available to the Frontend; the module does not create a separate JSON file for each request or expose its own HTTP endpoint. The current comparison dialog renders the objective matrix and evidence. The full personalized analysis is available in the module output but is not separately rendered in that dialog, as discussed in Section 5.4.

Status fields let the caller distinguish incomplete evidence from a failed execution:

Status	Meaning
empty	No products were selected
offline_preview	No provider was configured; direct supported facts are returned
partial	At least one model stage failed; available validated results are retained
completed	All requested stages returned without a caught error

A missing full description produces a warning while leaving the title, bullets and details available for analysis. A failed model call does not erase results already obtained from earlier stages. The caller can present the partial comparison and offer an explicit retry. `completed` describes execution and should not be interpreted as a factual-accuracy rating.

The module is stateless between calls. The Agent owns caching and invalidation using product IDs and order, requirements, supplied facts and provider identity. This keeps comparison generation separate from session management and lets the runtime reuse available output for repeated requests, export or finalization. Changes to the inputs require the runtime to reconsider reuse.

Provider identity, warnings and provenance help developers investigate an unexpected result. Usage and latency describe the reported execution of the module; they are not per-stage timing traces or a complete billing record. Failed provider requests may incur costs that are not returned in the response.

5.3.4 Evaluation and limitations

The eight committed regression tests were previously rerun successfully against 62b6925. They use simulated model responses to check the module's contracts and failure behavior. The cases cover separation of the model stages, rejection of unsupported attributes and fabricated quotes, offline unknown values, skipped personalization, empty or oversized selections, duplicate IDs, provider failure and generic schema fallback.

A controlled-preference case also checks that objective output remains unchanged when requirements change. It uses a deterministic fixture and holds the category hint constant. This supports the intended separation of objective information from personalization, but does not measure repeatability or semantic consistency with a live model.

The principal factuality limitation is that quote occurrence is weaker than entailment. A model can cite a real sentence while drawing an unsupported conclusion from it. The explicit/inferred labels require the same semantic scrutiny. Prompt instructions and reference checks constrain the output, but a live adversarial evaluation would be

needed to measure resistance to misleading catalog text or prompt injection.

Catalog completeness is another limit. Extraction cannot recover facts omitted from the handoff, and generic schema fallback does not establish broad category coverage. Sequential calls also add delay and provider cost. Offline and partial paths preserve usable information, but they do not provide the same analysis as a successful three-stage execution.

The next evaluation should use real selected-product sets with human-reviewed attributes and claims. Attribute precision and claim support rate would measure factual correctness; controlled requirement changes would test personalization consistency; usefulness ratings would assess whether the comparison helps a decision. Live median/p95 latency and token usage should be measured alongside these outcomes. These results remain unmeasured, so the present evidence supports the tested workflow and validation mechanisms rather than a claimed live-model accuracy or user benefit.

5.4 Frontend and user experience

Owner: Frontend.

The Frontend addresses the gap between receiving recommendations and making an inspectable choice. A conversation can capture changing needs, but shoppers also need to see which requirements are active, inspect the evidence behind each result and keep a small comparison set while continuing to browse. Shopping Copilot brings these tasks into one workspace, with progressively deeper views for product details and comparison. Its role is to present the Agent's decisions and collect explicit user actions.

5.4.1 Inputs and workflow

The implementation uses React and TypeScript, with Vite for development and bundling. The `useShopping` hook coordinates session requests and screen state; shared contracts describe products, requirements, selection state and comparison responses. On startup, the client checks `/api/health`, reads the service's data disclosure and creates a session through `/api/session`. Subsequent actions carry the returned session ID.

User action	Runtime interface	Screen update
Send a request, refine preferences or request more options	<code>/api/chat</code>	Update conversation, ranked products, requirements and selection state
Save, remove or clear options	<code>/api/select</code>	Refresh the shortlist and discard the previous comparison view
Inspect a product	<code>/api/product</code>	Open the detail drawer for its stable product ID
Compare saved options	<code>/api/handoff</code>	Open a comparison using returned product records and evidence
Confirm the saved selection	<code>/api/chat</code> with an explicit finalization message	Display the backend's confirmed selection state

The desktop workspace contains a conversation panel, a product list and a decision panel showing saved options alongside Must have, Preferences and Avoid. The shopper can inspect a result, save up to three options, compare at least two saved options and explicitly confirm the selection. A comparison returned by a conversational request can also open directly. The UI distinguishes requested-product comparisons from saved options; confirmation acts on the saved selection. Development requests use Vite's `/api` proxy to the local Python backend at port 8000.

5.4.2 Core methods

The interface uses progressive disclosure to keep the result list readable. Each product card shows its rank, available price and rating, requirement coverage, a short fit reason and up to two catalog excerpts. When generated fit text is unavailable, it uses an available deterministic reason or states that evidence is limited. The detail drawer exposes fuller listing text, attribute evidence, match statuses and cautions. The comparison dialog uses product IDs to align attribute values across columns and provides expandable source evidence. Entirely unlisted dimensions can be revealed with the Show unlisted details control.

Presentation helpers preserve the difference between missing data and actual values: an absent price becomes “Price unavailable,” an absent rating becomes “No rating,” and an unknown comparison cell remains “Unknown.” Zero is retained as a value. Numeric prices do not acquire an assumed currency, and the interface states that catalog data does not verify live availability. Product links are displayed only when they match the permitted HTTPS Amazon product-URL pattern; unverified links remain labelled and open in a separate tab with `noopener noreferrer`.

State-changing requests are serialized by the API client, while an interaction lock and disabled controls prevent overlapping actions in the UI. Screen state is updated from successful backend responses, and the message draft clears only after success. Requests have a 45-second timeout and are not automatically replayed. A timeout, network failure or unreadable response marks the session unusable because the client cannot confirm whether the action completed; the interface offers a new conversation. Expired sessions also trigger recovery. These mechanisms reduce ambiguous duplicate actions without claiming server-side idempotency or cancellation.

Responsive CSS changes the desktop columns into mobile Chat, Products and Compare views, with a horizontally scrollable comparison table. Native dialogs, labelled controls, status announcements, visible keyboard focus and reduced-motion rules support accessible interaction. The composer handles input-method composition and distinguishes Enter to send from Shift+Enter for a new line. These are implemented provisions whose usability still requires browser and assistive-technology testing.

5.4.3 Outputs and downstream use

The Frontend converts runtime responses into visible recommendations, requirements that can be revised through conversation, saved options and comparison cells with evidence. Products follow the Agent's returned ranks, and save/detail actions use stable IDs. The UI exposes Undo/Redo controls when the runtime advertises the relevant capability. If preferences or selection changes invalidate confirmation, the shopper receives a notice to review and confirm again. Confirmation is explicitly described as saving a decision within the session; no order is placed.

For comparison, the client prefers the objective matrix supplied under `comparison_assist`. When that matrix is absent, it builds a basic view from returned price, rating and requirement coverage. It distinguishes completed, partial and model-unavailable comparison states. The current dialog renders this matrix and its evidence, but does not separately render the module's full personalized fit reasons, cautions or cross-product trade-offs. Those richer fields remain an integration opportunity. Product-card fit reasons are a separate presentation path and should not be confused with the personalized comparison output.

5.4.4 Evaluation and limitations

Code inspection is based on `main` commit `1abc6c`. The committed Frontend suite contains eight presentation-helper tests and four API-client tests, plus four hosting-worker and packaging tests. The helper cases cover missing versus zero values, scalar and array requirements, reply-label mapping, product-ID-indexed comparison cells and the fallback matrix. API cases check request serialization, error propagation without replay, unreadable responses and native fetch invocation. Hosting cases cover static assets, application-route fallback, selected missing-API/write-request behavior and packaged file presence. These counts describe committed test cases; the Frontend suites and build were not rerun for this documentation update.

The repository supplies `npm test`, `npm run typecheck`, `npm run build` and `npm run test:sites`; the packaging test requires the build output. These checks do not establish successful browser-level completion of the shopping journey. A final evaluation should exercise search, save, compare and confirm on desktop and mobile, including

preference changes, partial comparisons, expired sessions and uncertain network outcomes. Task time, user error rate, accessibility conformance and browser compatibility remain unmeasured here.

TypeScript interfaces document expected payloads but do not provide runtime schema validation: the API client parses JSON and casts it to the requested type. The included hosting worker serves static assets and application-route fallbacks; production API routing must be configured separately. The earlier Agent integration measurements also show why the 45-second client timeout matters for cold searches. The implemented interface makes requirements, evidence and confirmation visible, while measured gains in decision speed or user confidence remain to be established.

6. Integrated execution example

The recorded local HTTP check on 62b6925 exercised a complete decision sequence: “I need a blue cotton dress” returned ten products from the local search index; “Compare #1 and #2” resolved the displayed IDs, compared those products and saved them in that scenario; “Finalize my selection” changed the shortlist state to `finalized`. The explicit final turn was the decision checkpoint. This run used cloud assistance off, so its comparison demonstrates the offline path rather than live generation.

A separate recorded five-turn detail-follow-up run preserved product identity and requirements through reference clarification, preference recap and a care question. Missing care instructions remained unknown. This illustrates continuity beyond a single search; broader corrections and failure behavior are covered by regression fixtures, not by this short HTTP sequence alone.

7. Integration contracts

Boundary	Payload and validation
Frontend → Agent	Session-scoped JSON requests for chat, product details, selection and handoff; client serializes mutations
Agent → Search/Ranking	<code>search_products(query, top_k)</code> and <code>get_product_details(product_ids)</code> ; adapter checks IDs and finite scores
Internal retrieval/ranking	<code>RetrievalResultV2</code> and <code>RankingResultV2</code> carry matching session, turn and state version; limits and ranks are checked
Agent → Description/Comparison	<code>show-me-your-agent.selection.v1</code> : ordered <code>selected_products</code> , unique nonempty IDs, maximum three, and <code>hard/soft/excluded</code> requirements
Description/Comparison → Agent	<code>description-comparison.v1</code> : profiles, objective and personalized results, status and provenance; attached under <code>comparison_assist</code>
Agent → Frontend	Assistant message, ranked products, receipt, selection state and optional comparison handoff; current UI coverage is described in Section 5.4

8. Evaluation and decision readiness

The table separates measured ranking quality, tested software behavior and remaining evaluation work. User-effort and business improvements have not yet been measured.

Capability	Available evidence	Interpretation and remaining gap
Relevance	Reported official ESCI US test: 8,956 queries, 181,701 pairs; full-list nDCG 0.859077 → 0.867305	Reranking over supplied candidates; full-catalog recall is not measured by this result
Continuity and control	615 isolated regressions passed on 1abcb6c	Tested state, references, recovery and integration; not a language-understanding accuracy score
Comparison contracts	Eight tests previously passed on 62b6925 with simulated model responses	Stage separation, citation rejection, unknowns, selection validation, failure handling and generic fallback
Preference separation	Deterministic fixture preserves objective output when preferences change, with category hint fixed	Live-model consistency and claim entailment remain unmeasured
Complete local journey	Recorded real-search lifecycle and detail follow-ups on 62b6925, cloud assistance off	Confirms that recorded local path, not the live generative pipeline
Interface actionability	Committed helper/API/hosting cases described in Section 5.4	Browser task completion, accessibility and user-effort gains remain unmeasured

Failure cases cover invalid references, duplicate or oversized selections, unsupported quotes, provider errors and missing evidence. Live adversarial catalog evaluation remains outstanding. The recorded cold search exceeded the Frontend's 45-second timeout; timings and experimental conditions are retained in Section 5.2.4.

9. Human control, security and observability

The Agent can retrieve information and propose bounded clarification, while shoppers resolve ambiguous intent, choose comparison products and explicitly confirm their saved selection. Requirement or shortlist changes can invalidate confirmation. Undo/Redo provides recovery for supported edits. No tool exposes payment, ordering or reservation; staff escalation and production authorization remain future integration.

Catalog text is untrusted input. Allowlisted factual fields and source-quote checks constrain extraction and comparison; DeepSeek requirement edits are validated as a batch, and response writing validates product and evidence IDs. The interface also restricts external product-link patterns. Evaluation boundaries for these checks are described in Sections 5.3.4 and 8.

Capability separation keeps state mutation in the Agent and comparison analysis in a bounded module. It is a software boundary, not an operating-system sandbox. Backend credentials, service permissions, session ownership and retention require deployment controls. Cloud assistance is explicitly configured and can send shopper messages, requirement state and catalog excerpts to the provider; the runtime supplies disclosure. The documented implementation does not establish production least-privilege isolation.

Receipts and traces expose requirement evidence, state versions, policy decisions, product IDs, status and model usage. Comparison adds provider identity, warnings, provenance and total latency. Agent trace `elapsed_ms` values are offsets from the turn start, not independent stage durations. The `show-me-your-agent.audit.v1` export uses an unsigned SHA-256 chain: it can detect inconsistent records, but can be recomputed and is not an authenticated tamper-proof ledger. This supports inspection of executed actions without relying on private model reasoning.

10. Platform and reproduction

Python coordinates explicit state, tools and the optional generation provider; `bm25s` provides bounded lexical retrieval; the fine-tuned Cross-Encoder supplies joint query-product scoring. React/TypeScript separates session interaction from rendering, and Vite provides the development proxy and build. Each technology serves the module boundary described above; generative assistance is off unless configured.

The public demo at 18.138.181.132 serves the Frontend and `/api` through the same HTTP address.

From the repository root, with the required Python environment installed:

```
python -B -m agentic_workflow.verify
python3 -B -m unittest discover -s description_module/tests -v
python -B -m agentic_workflow.preflight
python -B -m agentic_workflow.live_smoke --detail-followups
python -B -m agentic_workflow --host 127.0.0.1 --port 8000
```

The first two commands exercise isolated tests; preflight, live smoke and normal search need the local index, matching model and search dependencies. `--demo` explicitly selects synthetic products. Full comparison generation additionally needs a configured provider; the standalone comparison demo produces an offline preview.

In frontend, install locked dependencies with `npm ci`, then use `npm run dev` for the local UI. Validation commands are `npm test`, `npm run typecheck`, `npm run build` and, after building, `npm run test:sites`. These commands are reproduction guidance, not a claim that a fresh-clone build or production deployment was verified in this edit. The static hosting worker does not supply production API routing. Shared session storage, access control, cold-start readiness and load testing remain deployment work.